

Lesson 6: Debugging

45 minutes

Overview

In this lesson students practice using the different programming concepts that they were introduced to in the last lesson. To begin, however, they are introduced to the concept of debugging and are encouraged to use and reflect on this practice throughout the lesson. At the end of the lesson students share their experiences debugging as well as an new realizations about programming.

Standards

Full Course Alignment

CSP Conceptual Framework

- **AAP-2** - The way statements are sequenced and combined in a program determines the computed result. Programs incorporate iteration and selection constructs to represent repetition and make decisions to handle varied input values.
- **CRD-2** - Developers create and innovate using an iterative design process that is user-focused, that incorporates implementation/feedback cycles, and that leaves ample room for experimentation and risk-taking.

CSTA K-12 Computer Science Standards (2017)

- **CS** - Computing Systems

Agenda

Warm Up (5 minutes)

Preview the Lesson

Activity (30 minutes)

Practice Time

Wrap Up (10 minutes)

Assessment: Check For Understanding

Objectives

Students will be able to:

- Debug simple sequential and event-driven programs
- Use the debugging process and Identify specific best practices for debugging programs
- Use the speed slider, break points, and documentation as part of the debugging process

Preparation

- Review the steps of the debugging process
- Review the **Debugging** one-pager
- Review the levels students will need to complete on Code Studio
- Check the **"Teacher's Lounge"** forum for verified teachers to find additional strategies or resources shared by fellow teachers
- If you are teaching virtually, consider checking our **Virtual Lesson Modifications**

Links

Heads Up! Please make a copy of any documents you plan to share with students.

For the teachers

- **Debugging** - Resource [▼ Make a Copy](#)
- **Debugging** - Slides [▼ Make a Copy](#)

For the students

- **Class Chats: Olawale Oladehin** - Video ([Download](#))

- [How To Debug](#) - Video ([Download](#))

Vocabulary

- **Comment** - form of program documentation written into the program to be read by people and which do not affect how a program runs
- **Debugging** - Finding and fixing problems in an algorithm or program
- **Documentation** - a written description of how a command or piece of code works or was developed

Teaching Guide

Warm Up (5 minutes)

Preview the Lesson

 **Discuss:** *Your friend calls and says "I can't get music to come out of my speakers, can you help?". Write a quick list of everything you'd ask them or have them check to try to fix the problem.*

Discussion Goal: This prompt should help get students in the mindset of debugging. If students ask for more details tell them that this is all the information they have and that they should feel free to come up with as many questions as they think would help them get the information they need to be helpful.

Remarks

Today we're going to practice programming, but we're also going to practice a very important skill in programming called "debugging". Let's see what it looks like.

Activity (30 minutes)

Practice Time

 **Display:** Show the [Debugging video](#) and then show the debugging process.

Teaching Tip

Videos are used throughout the curriculum to spark discussions, supplement key concepts with additional explanations and examples, and expose students to the various roles and backgrounds of individuals in computer science.

While interacting with the video, turn on closed captioning so students can also read along as they watch.

To encourage active engagement and reflection, use one or more of the strategies discussed in the [Guide to Curriculum Videos](#).

Remarks

We're going to use this process to help us fix programs. I want you to use this process as you fix issues you find in code today. At the end of the lesson we'll talk more about this process and any specific strategies you document along the way.

Group: Place students in pairs.

Level 1: This level requires students to recognize that strings need to go in quotes.

Level 2: This level also requires students to recognize that strings need to go in quotes. Ideally students will get more familiar with checking for yellow warnings in this level.

Level 3: This code has no warnings but in testing the app students should see that the range of values for the random numbers is too large. Iterative testing with different values should help them pick a number that keeps the smiley inside the screen.

Level 4: This level explicitly introduces the fact that event handlers (**onEvent**) should not go inside other **onEvent** blocks. This is explicitly addressed in the concluding slide show as well.

Level 5: This predict level asks students to hunt for a bug and give advice to the programmer to avoid bugs of this type in the future. This level emphasizes different parts of the Debugging process but especially the *Document* step, getting students to consider:

- *What have you learned?*
- *What strategies did you use?*

Assessment Opportunity

Use this question to help formatively assess your students and gauge their use of the debugging process.

For more information, check out the [CSP Guide to Assessment](#).



1-5

Debugging

1

2

3

4

5



Level 6: Have students complete at least one of the levels in this choice level

- **6a** - The code in this level will run, but the wrong text and sounds have been placed in the wrong places. This shows that code may have no warnings but still is not working correctly.
- **6b** - In this level, an entire event handler is missing. Students will need to add it to the app.
- **6c** - This app is a good chance for students to practice adding functionality of their own. While nothing starts off as "broken", students will need to use debugging practices as they add code to this app.



6

Practice



Teaching Tip 

Choice Level: Students tend to be more engaged and respond better when they have an authentic choice about how to continue their learning. Allow students to choose practice levels according to their interests and level of comfort.

Wrap Up (10 minutes)

 **Journal:** Have students add the following vocabulary words and definitions to their journals: documentation and comment.

 **Discuss:** *Create an analogy in which you compare and contrast the process of debugging with different actions / events / things from your life.*

Discussion Goal: This discussion should help reinforce the fact that debugging is a skill that can be learned and that it is made up of many little steps and understandings.

 **Do This:** Play the music clip to cue the Give One, Get One activity and direct students to participate in a Give One, Get One.

Teaching Tip

Using music to cue transitions helps create routines in the classroom. When students hear cues, they know it is time to transition to a specific activity. Emphasize to students what the music clip means, and be consistent with the use of the music clip to cue the activity.

Click the blue play icon on the slide to play the music cue.

Give One, Get One encourages students to seek and share ideas and information. Students circulate the classroom and pair up with a partner when prompted by the teacher. Each student "gives" or shares one of their ideas as the other student "gets" or listens and writes it down. The teacher then prompts students to find new partners and repeat the process.

 **Display:** Show the slide which lists some best practices in addition to those students may have mentioned.

Remarks

Debugging is an important and entirely normal part of programming. Your code won't always work the first time, and that's OK! Debugging is a skill that you can practice and get better at. Using documentation and leaving comments for yourself are important skills, but so is working with classmates or learning to more effectively search for bugs. We'll keep using these skills.

Teaching Tip

Class Chats Video If time allows, consider showing [this clip](#) of Amazon Web Service's Olawale Oladehin discussing the value of creating bugs on purpose!

Remarks

Let's take a minute to talk more about comments and documentation. Comments help explain the code, but do not affect how the program runs. They are meant to be read by people! When we write code, we don't only write for the computer we also write for other people. It's important that others can understand our code, so write your code clearly using the practices we discussed and comments.

Not all programming environments support comments, so other forms of documentation may be important like keeping a separate document with information about your program. The key takeaway here, is no matter what the format, comments and documentation are important!

As you grow in your programming skills, you will start to appreciate how valuable comments can be. You don't have to wait until a program is complete to write the comments. You should be doing this as you develop the project. There will be opportunities to write comments called out in the App project you are currently working on.

Assessment Opportunity

Create Performance Task Written Response Practice

In order to help prepare students for the new component of the AP Create Performance Task, there will be several opportunities throughout the curriculum to practice creating written responses aligned with the learning objectives that might be assessed on the exam.

Create PT Learning Objective(s):

- Identify the error.
- Correct the error.

Unit Journal Template You may choose to reuse this debugging journal template if you think it will help support your students becoming stronger debuggers.

Sample Response for Instructors

Assessment: Check For Understanding

Check For Understanding Question(s) and solutions can be found in each lesson on Code Studio. These questions can be used for an exit ticket.

- What is one thing you really enjoyed about today's activity?
- Is there anything that you found confusing or need extra help with?



Check for Understanding



This work is available under a **Creative Commons License (CC BY-NC-SA 4.0)**.

If you are interested in licensing CodeAI materials for commercial purposes **contact us**.